Web Application Penetration Test Report

Target: TryHackMe – “Root Me” Room

Prepared by: Manoj Bora

Date: 15 September 2026

Classification: For Academic / Training Use Only

Document Control

Report Title	Web Application Penetration Test – TryHackMe Root Me
Target	http://10.48.191.240 (TryHackMe lab instance)
Tester	Manoj Bora
Engagement Type	Black-box, authorized training lab (TryHackMe)
Date of Testing	15 September 2026
Date of Report	15 September 2026
Classification	For Academic / Training Use Only

Executive Summary

This report covers the TryHackMe “Root Me” room, completed as part of the web module of my NPT (Network Penetration Testing) practical coursework. I found a chain of two connected issues on the target: a file upload feature that accepted a working PHP webshell once the extension was changed to get past its filter, and a SUID-flagged Python binary on the box that let a low-privilege shell jump straight to root.

Using the webshell for initial access and the SUID binary for privilege escalation, I was able to go from an anonymous, unauthenticated visitor to root on the machine and capture both flags — user.txt (THM{y0u_g0t_a_sh3ll}) and root.txt (THM{pr1v1l3g3_3sc4l4t10n}). The full technical details and risk ratings for each issue are covered in the Findings and Risk Summary sections below.

Flags Captured

Flag	Value
User Flag (user.txt)	**THM{y0u_g0t_a_sh3ll}**
Root Flag (root.txt)	**THM{pr1v1l3g3_3sc4l4t10n}**

Scope & Objectives

Target: TryHackMe machine at 10.48.191.240.

Type of test: black-box — no credentials or information were given beforehand; everything was found during testing.

Objective: identify and exploit web application vulnerabilities to gain a foothold on the target, then escalate privileges to root, as required for the web module practical.

Methodology

The assessment followed the standard CEH five-phase approach, scoped down to what was actually needed for a single web target:

- Reconnaissance – identifying the target and the services it exposes.
- Scanning & Enumeration – port scanning and directory/file brute-forcing against the web server.
- Gaining Access – exploiting the file upload feature to get a reverse shell.
- Privilege Escalation – identifying and abusing a misconfigured SUID binary to go from a web-server shell to root.
- Maintaining Access / Covering Tracks – not applicable to this engagement; the objective was flag capture, not persistence.

1. Reconnaissance & Enumeration

I started the room and was given a target IP. A TCP SYN scan was run with Nmap to identify what services were exposed before touching anything in a browser:

```
nmap -sS 10.48.191.240
```

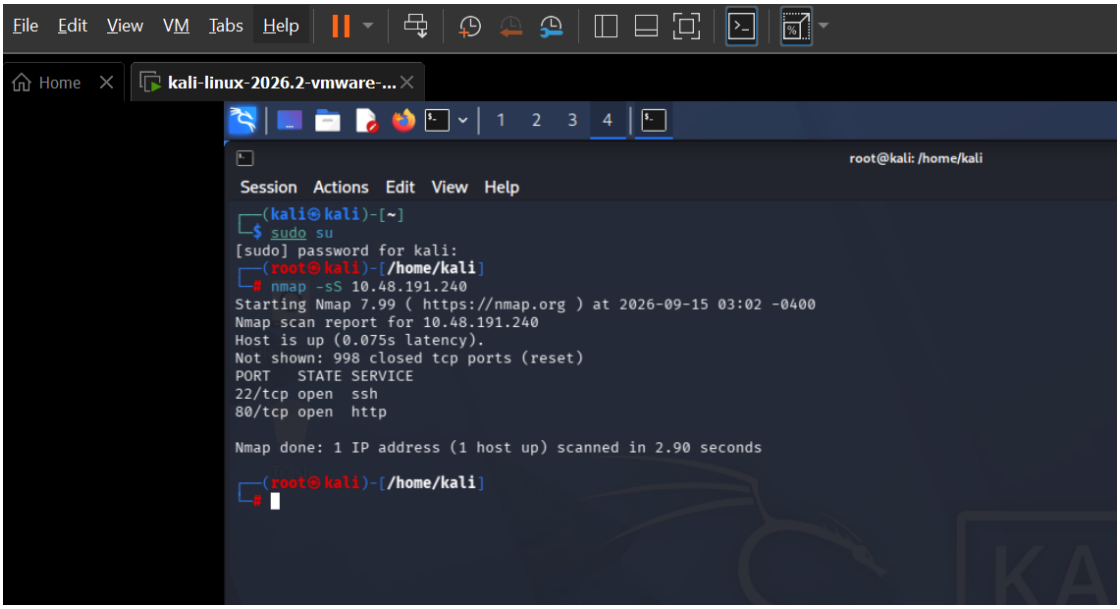


Figure 1 – Nmap scan showing SSH (22) and an HTTP service (80) open on the target

The scan showed two open ports: 22/tcp (SSH) and 80/tcp (HTTP). Browsing to the site directly didn't turn up anything useful, so I ran a directory brute-force with Gobuster against the web root:

```
gobuster dir -u http://10.48.191.240 -w /usr/share/wordlists/dirb/common.txt
```

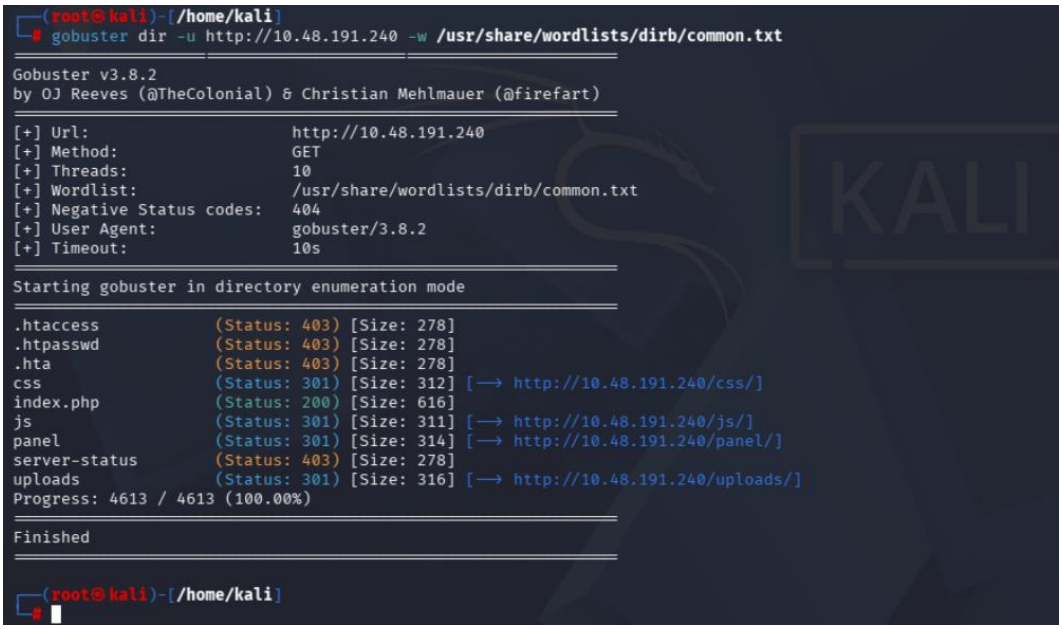


Figure 2 – Gobuster enumeration revealing /panel/ and /uploads/, among other directories

The scan turned up several directories, including css/, js/ and a protected server-status, but the two that stood out were /panel/ and /uploads/.

Tools Used

Tool	Type	Purpose
Nmap	Network scanner	Port scanning to identify exposed services on the target host
Gobuster	Directory brute-forcer	Enumerating hidden directories and files on the web server
Firefox (Kali)	Browser	Manual walkthrough of the upload panel and uploaded files
php-reverse-shell.php (pentestmonkey, Kali webshells)	Webshell payload	Gaining a reverse shell once uploaded and executed
Netcat (nc)	Listener	Catching the reverse shell connection
find -perm -4000 / GTFOBins	Linux privilege-escalation reference	Locating and abusing a misconfigured SUID binary

Findings

I found two connected issues during this room. The first gave initial access to the box as a low-privilege user; the second took that access straight to root. Severity ratings and CVSS scores for both are in the Risk Summary section at the end.

Finding 1: Unrestricted File Upload Leading to Remote Code Execution

The /panel/ directory found during enumeration led to a file upload form with no login required. I copied Kali's pentestmonkey php-reverse-shell.php webshell, edited it with my TryHackMe tun0 IP and port 1234, and tried uploading it as-is — the upload was rejected. Renaming the file to a .phtml extension and re-uploading is what actually got past the filter and left a working webshell sitting in /uploads/.

What I did:

- Opened http://10.48.191.240/panel/ directly — the upload form loaded with no authentication.
- Copied /usr/share/webshells/php/php-reverse-shell.php and set the IP to my TryHackMe tun0 address, port 1234.
- Uploaded the file with a .php extension — rejected by the form's filter.
- Tried a .php3 extension and renamed the file to php-reverse-shell.phtml, then uploaded again — accepted.
- Confirmed the file was live by browsing to /uploads/, where php-reverse-shell.phtml appeared in the directory listing.

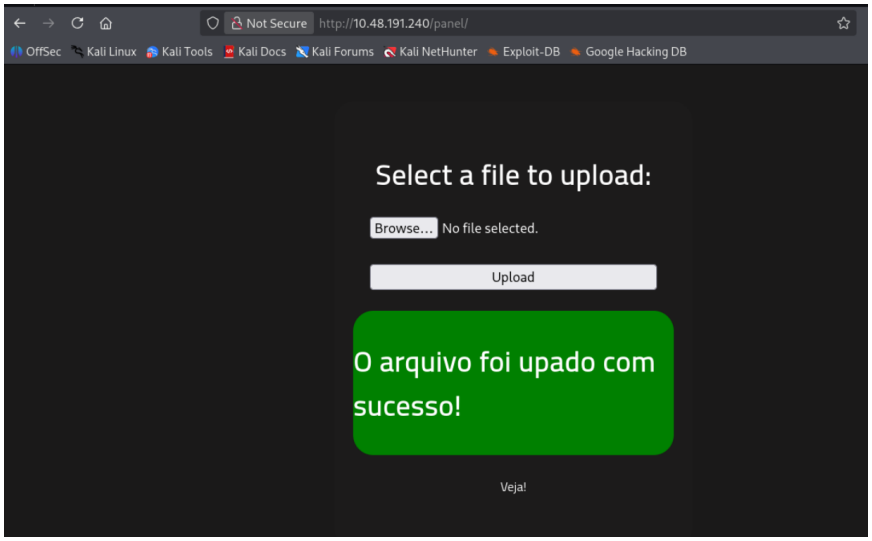


Figure 3 – The /panel/ upload form, accessible without authentication, confirming a successful upload

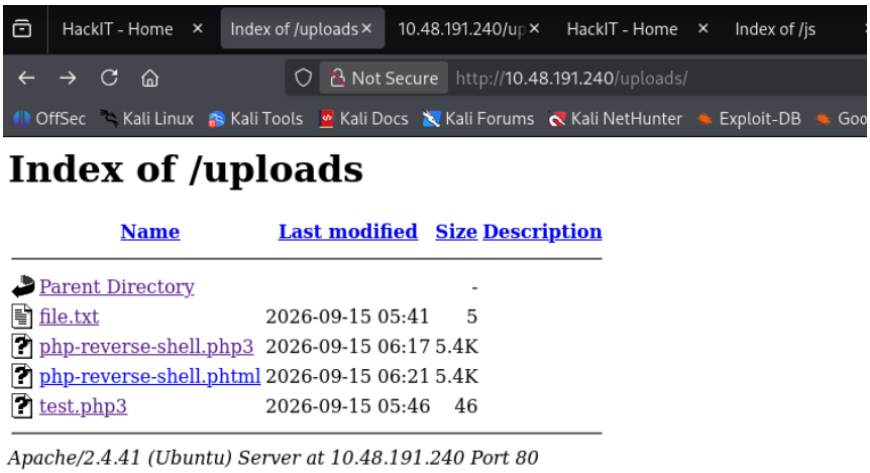


Figure 4 – The /uploads/ directory listing showing the .php3 and .phtml files that got past the extension filter

With the webshell in place, I started a Netcat listener and requested the .phtml file in the browser to trigger the callback:

```
nc -lvnp 1234
```

Browsing to http://10.48.191.240/uploads/php-reverse-shell.phtml triggered the shell, and the listener caught a connection back as the www-data user:

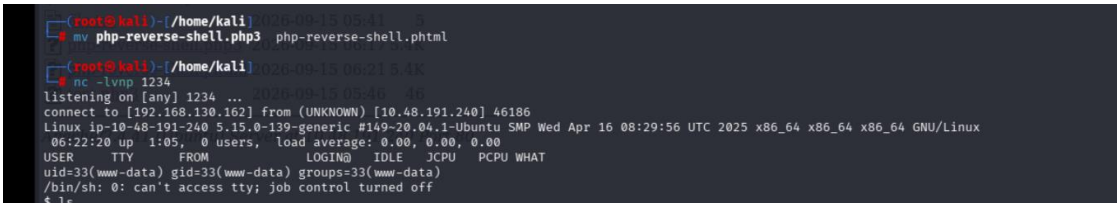


Figure 5 – Renaming the payload to .phtml and catching the reverse shell as www-data

From the shell, I searched for the first flag and found it under the web root:

```
find / -type f -name user.txt 2>/dev/null
cat /var/www/user.txt
```

This returned the first flag: THM{y0u_g0t_a_sh3ll}

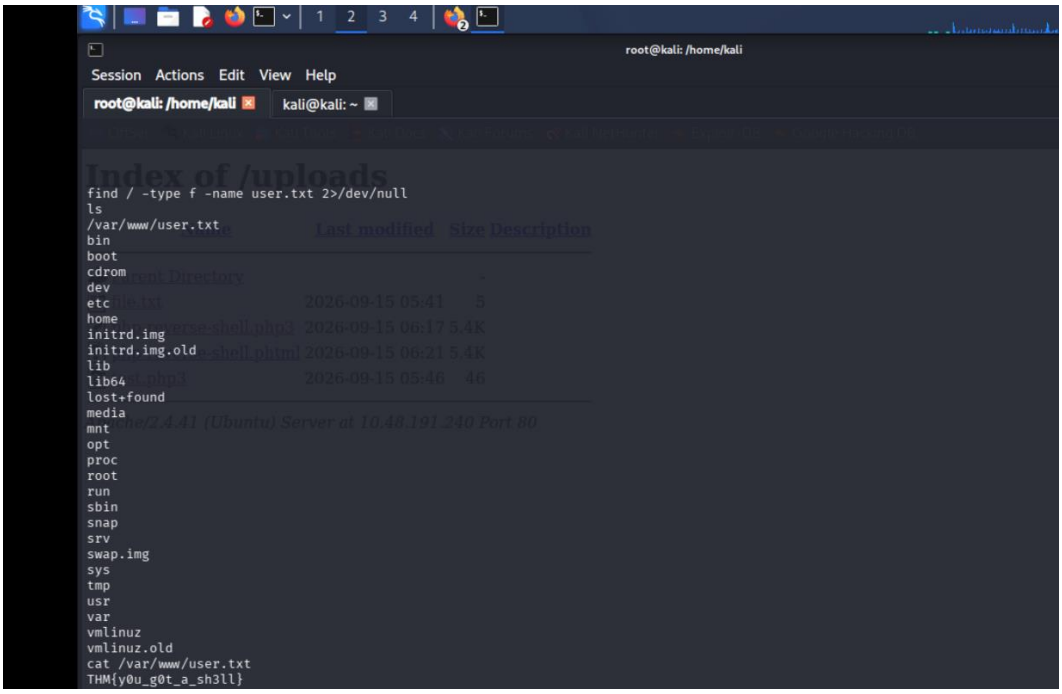


Figure 6 – Locating and reading user.txt for the first flag: THM{y0u_g0t_a_sh3ll}

Why it matters: The upload form only blocked the plain .php extension. Apache's PHP handler on this box was still configured to execute alternate PHP-associated extensions like .php3 and .phtml, so a one-word rename was enough to bypass the filter entirely and get arbitrary code execution on the server.

How to fix it: Validate uploads with an allowlist of safe extensions and MIME types rather than a blacklist, don't let the web server execute any file inside the uploads directory, and require authentication before this kind of functionality is reachable at all.

Finding 2: Privilege Escalation via Misconfigured SUID Binary (python2.7)

Once I had a shell as www-data, I enumerated the box for privilege-escalation paths, starting with SUID binaries:

```
find / -perm -4000 -type f 2>/dev/null
```

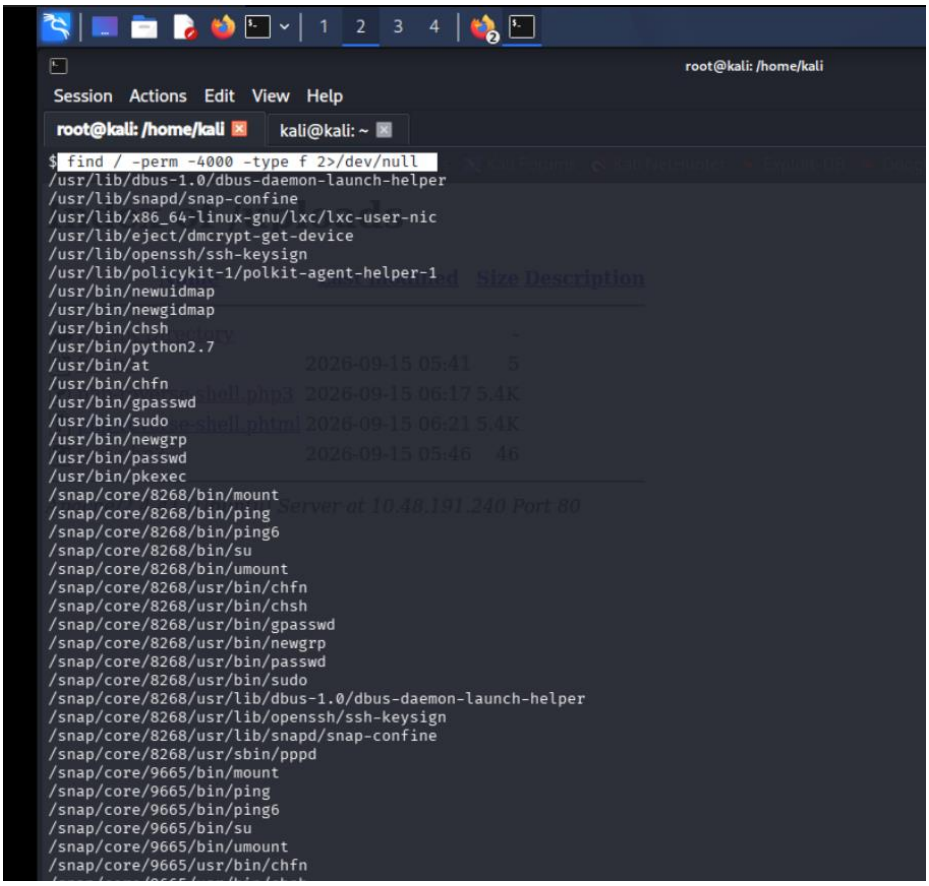


Figure 7 – SUID binary enumeration; /usr/bin/python2.7 stands out among the results

/usr/bin/python2.7 was in the list, which isn't a default SUID binary. I checked its permissions to confirm the SUID bit was actually set:

```
ls -la /usr/bin/python2.7
-rwsr-xr-x 1 root root 3657904 Dec 9 2024 /usr/bin/python2.7
```

Since python2.7 has a well-known GTFOBins privilege-escalation technique, I used it to spawn a shell that inherits the binary's root ownership:

```
python2.7 -c 'import os; os.execl("/bin/sh", "sh", "-p")'
```

This dropped me into a shell running as root. From there I located and read the final flag:

```
find / -type f -name root.txt
cat /root/root.txt
```

This returned the final flag: THM{pr1v1l3g3_3sc4l4t10n}



Figure 8 – Using the SUID python2.7 binary to spawn a root shell and reading the final flag: THM{pr1v1l3g3_3sc4l4t10n}

Why it matters: Giving the SUID bit to a general-purpose interpreter like Python is effectively equivalent to giving out root — the interpreter can be told to execute any command or spawn any shell, and it will do so with the privileges baked into the SUID bit rather than the calling user's own privileges.

How to fix it: Remove the SUID bit from interpreters and any other binary that doesn't specifically need it (chmod u-s), and periodically audit the system with find / -perm -4000 as part of routine hardening. Where an interpreter genuinely needs elevated rights for one task, use a narrowly scoped wrapper or sudoers rule instead of a blanket SUID bit.

Risk Summary

Finding	CVSS 3.1 Base	Assessed Severity	Category
Unrestricted File Upload → Remote Code Execution	9.8 (Critical)	Critical	CWE-434 Unrestricted Upload
Privilege Escalation – Misconfigured SUID Binary	7.8 (High)	High	CWE-269 Improper Privilege Mgmt

Conclusion & Recommendations

The Root Me machine could be taken from an anonymous, unauthenticated visitor to root using nothing more than a weak file-extension filter and a Python binary that should never have had the SUID bit set. Both steps were quick once identified, and neither needed anything beyond commonly available tools.

What should be fixed first:

- The SUID bit on /usr/bin/python2.7 (Finding 2) — this is what turned a low-privilege webshell into full root.
- The file upload filter (Finding 1) — this is what let an unauthenticated user get code execution in the first place.

Overall, the box's security comes down to trusting the wrong things twice: trusting an extension blacklist to be complete, and trusting an interpreter with a root-level SUID bit it never needed. Fixing either one on its own would have broken this chain.

What I learned: this room was a good reminder that upload filters need to be allowlists, not blacklists — blocking one extension while missing its siblings defeats the whole point of the filter. It also reinforced that privilege escalation is often just a matter of systematically checking the obvious things, like SUID binaries, rather than anything exotic.